PACN

A Python package for controlling light strings

Introduction

A standard method for controlling light strings is the “streaming Automated Control Network” protocol (or sACN), defined officially by a document called E1.31. PACN provides a partial implementation in Python.

The standard defines how a host computer transmits data over a network to some sort of receiver device. The receiver is responsible for handling the data and producing the physical output. A typical receiver provides electrical power for light strings as well as the logic that distributes the data to the individual lights.

A program on the host produces data, formats it according to the standard, and transmits it by UDP on a network. In order for this to work, the host program must know exactly what the receiver will do with the data. That is not defined by the standard.

In order to make sense of the code in this package, it’s necessary to dip briefly into some terminology and some of the rationale of the E1.31 standard.

E1.31 in Brief

UDP senders and receivers

In E1.31, network data is sent and received as UDP packets. This is a one-way mechanism that sends data from host to receiver. It’s “unreliable” in the sense that the host has no ability to know if the data got delivered. The receiver has no ability to know if it missed something. To address this, the standard requires the host to transmit periodically all the data, regardless of whether it has changed. If the receiver hasn’t heard from the host recently, it’s not required to “leave the lights on.” When data transmission stops, the state of the hardware is undefined.

The definitions of “periodically” and “recently” in the previous paragraph have a fairly wide range. The standard specifies that the data must be refreshed every 10 seconds, but some receivers (allegedly and by personal experience) will not wait nearly this long. I have observed a timeout interval as short as 0.2 seconds. Therefore, the PACN implementation uses 0.1 seconds as the refresh interval. All data items are re-transmitted on that timescale, even if they are unchanged. If that seems ridiculous to you, well, it does to me, too. Tinker with it if you are so inclined.

Terminology – lights, pixels, and universes

The standard describes how each UDP data packet is constructed. It consists of two main parts.

First there is header - a sequence of bytes that identifies a packet as E1.31-compliant. E1.31 receivers will only respond to packets that have this header; the PACN code takes care of constructing the headers.

The second part is the payload – the data that, you hope, will make the lights change color. This is where things get interesting. As already mentioned, the standard says nothing about what the receivers will actually do with the data. The only way to know what a specific receiver is going to do is to rely on some other source of information, such as the receiver's documentation or (worst case) some DIY hobbyist's YouTube video.

That said, I'm now going to talk specifically about a receiver that controls strings of *colored* lights. These lights are closely analogous to pixels in digital images, and for the rest of this write-up I will call them "pixels." The brightness and color of a pixel is specified by three numbers, representing the amount of each of the three primary colors: red, green, and blue. Each value is an integer ranging from 0-255 and fits into a single 8-bit byte of data. The higher the number, the more intensity. Therefore [0, 0, 0] is black, [255, 255, 255] is white, [127, 127, 127] is middle gray, [255, 0, 0] is pure red, [255, 0, 255] is magenta (red plus blue), and so on. This method is consistent with digital cameras and much digital display hardware.

In the real world, a pixel might physically consist of several LEDs that work together as a unit to produce the desired RGB effect. To the programmer this detail doesn't matter. Each hardware unit functions as one pixel represented by three bytes of data. Some documentation and, especially, marketing literature, can be confusing in its use of words like "lights," "LEDs," and "pixels." The rule I'm going to use here at PACN Project Headquarters is: **a pixel is a thing that is controlled by three RGB bytes**.

The E1.31 standard organizes data into structures it called "universes," which sounds awesome but turns out to be just a weird name for a sequence of bytes. Each universe has an integer "universe number" that distinguishes it from other universes. The only limit is that all of the universes on a given network must have different universe numbers. Universe numbers can range from 1-63999, so you're not going to run out.

The standard places a limit of 512 bytes on the size of a universe. This means that a universe can contain 170 pixels at most, as you can verify by dividing 512 by 3. Many light strings are shorter than 170 pixels, which does not present a problem. I recommend assigning a different universe number for each string in those cases, but this is not required. If the hardware supports multiple strings per universe, you are free to assign the universe numbers any way you want.

Some light strings, on the other hand, have more than 170 pixels. For those long strings, multiple universes will be needed to address all the lights on the strings. How the universe numbers map to the strings is completely up to the receiving hardware, which must be properly configured for everything to work correctly.

Executive summary:

- 3 bytes = data for 1 pixel (no matter how it's done in the hardware)

- Universes hold up to 512 bytes (< 171 pixels)
- Universes have unique integer indexes (1-63999)
- Mapping the Universes to the lights is entirely the responsibility of the receiver hardware.

PACN Implementation

Two classes are of prime importance in PACN.

- A **Universe** represents the hardware state of one Universe.
- A **Multiverse** manipulates the **Universes** and insures that their data is transmitted properly over the network.

A typical application does these steps:

- Create a Multiverse.
- Create one or more Universes, and register them with the Multiverse.
- Produce the desired light show by modifying the universe contents over time.

Example Programs

The PACN package includes three example programs, illustrating three methods of producing real-time lighting effects. All three programs use the same light setup, four light strings that represent four "Candy Canes." The goal is to make a Christmas-themed display where lights patters travel up the strings. The light pattern consists of a sequence of white pixels alternating with a sequence of color pixels, either red or green.

Simulation program

To visualize this without actual hardware, PACN includes a test program called e131_sim.py. This program runs on the local network, listens for E1.31 packets, and uses Python's standard tkinter GUI to make a simple on-screen display of the data it receives. You can run this program from the command line.

The example programs transmit data to 127.0.0.1, which is the standard "loopback" IP address. So if you run the example programs from a different command line on the same computer, the e131_sim program will show you the results.

To make an actual light show, replace '127.0.0.1' with the real IP address of your hardware.

Example 1: Update the light strings in a callback function

```
from pacn.multiverse import Multiverse
from pacn.universe import RGBUniverse
from pacn.utils import DEFAULT_CID_BYTES
from pacn.pac_logger import logging

IP = '127.0.0.1'   # Loopback for use with e131_sim.py
```

```
logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)

canes: list[RGBUniverse] = []

def create_canes() -> None:
    wide = ["red"] * 6 + ["white"] * 9
    narrow = ["green"] * 4 + ["white"] * 11
    size = 150
    stripes = size // len(wide)
    patterns = [wide, narrow, narrow, wide]
    for i, p in enumerate(patterns, 1):
        cane = RGBUniverse(DEFAULT_CID_BYTES, i, IP, size)
        cane.set_pixel_data(p*stripes)
        canes.append(cane)

def step(_t: float) -> None:
    for c in canes:
        pixels = c.pixels()
        pixels = pixels[-1:] + pixels[:-1]
        c.set_pixel_data(pixels)

def main() -> None:
    create_canes()
    with Multiverse(callback=step) as mv:
        for c in canes:
            mv.add_universe(c)
        mv.wait_for_kb()

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        pass
```

Explanation

I'll skim over a lot of the first section, which is mostly imports and some boilerplate. The code creates a list of Universes in the variable **canes.** The function **create_canes** fills them with starting light patterns: alternating short runs of red and white pixels in the outer strings, green and white in the middle strings.

The important step is the constructor call:

```
RGBUniverse(DEFAULT_CID_BYTES, i, IP, size)
```

> **RGBUniverse** is a subclass of **Universe**, and is almost always more convenient. **Universe** manipulates data at the most primitive level, as bytes. Each byte is one of the three RGB color channels, so 3 bytes represent a single pixel's color. **RGBUniverse**, on the other hand, manipulates data as *color strings*. Each pixel is represented by one string. The format of these strings meets the CSS3/HTML standard, where the string can be a descriptive name ('black', 'white', 'green', 'magenta', etc.), a 6-character hexadecimal value preceded by a pound sign ('#000000', '#ffffff', '#00ff00', '#ff00ff', etc.), or the 3-character variant ('#000', '#fff', '#0f0',

‘#f0f’, etc.). Further information can be found in the file colors.py, which also serves as a reference for all of the descriptive names that PACN understands. The definition is:

```
class RGBUniverse(Universe):
    def __init__(self, cid: bytes, n: int, ip: str, pixel_count: int,
                 **kwds) -> None:
        """
        cid: identifier required by E1.31
        n: Universe number
        ip: network address of the PixelController or equivalent
        pixel_count: number of pixels in the universe
        kwds: the only valid keyword is source_name, a string identifying
              the program.  It has no signifance and defaults to "PACN".
        """
```

RGBUniverse initializes its data to fully off (all pixels are ‘black’). This program wants to initialize the strings to patterns of either red-and-white segments or green-and-white segments. The state of the lights in an RGBUniverse is set by calling the function set_pixel_data:

```
    def set_pixel_data(self, pixels: list[str]) -> None:
        """
        pixels: a list of strings defining the state all of the pixels.
        """
```

The last step in create_canes adds each of the four canes to the global list.

Skipping over the function step for the moment, here is the main function:

```
def main() -> None:
    create_canes()
    with Multiverse(callback=step) as mv:
        for c in canes:
            mv.add_universe(c)
        mv.wait_for_kb()
```

After creating the canes, the program instantiates a **Multiverse** instance. The Multiverse can manage multiple universes, handling all the details of network transmissions and ensuring that the light strings stay in sync with the data. The intention is that a program has only one Multiverse. It is most conveniently used as a context manager; when the context manager exits, it sets all the lights to off (‘black’) before shutting down the network connection.

The Multiverse constructor:

```
class Multiverse:
    def __init__(self, *, callback: Callable[[float,], None] | None = None,
            fps: int = 30, handle_sigterm=True) -> None:
        """An object to manage multiple universes.
        The Universes can be updated at a rate given by fps (frames per second).
        callback: a Callable taking one float, the current time.  If this
            parameter is set, the Multiverse will call it at the frame rate.
        fps: frames per second.  The default is 30. PACN has been tested at 50.
        handle_sigterm: blanks all the lights in all the Universes on
            exit.  Defaults to True.  If set to False, the light strings remain
            in their last state when the Multiverse exits.  The effect that
            results is hardware-dependent.
        """
```

The main() function continues by registering each Universe with the Multiverse. This is a necessary first step and is done by calling **add_universe**. The final step is a call to **wait_for_kb**, which blocks the main thread until the user presses Control-C or the operating system sends a signal to shut down the program. The Multiverse continues to do its work in a secondary thread.

In Example 1, the Multiverse is instantiated with a callback function, **step**, which I skipped earlier.

```
def step(_t: float) -> None:
    for c in canes:
        pixels = c.pixels()
        pixels = pixels[-1:] + pixels[:-1] # the last pixel is now first
        c.set_pixel_data(pixels)
```

This function gets called at the Multiverse frame rate (30 times per second). It runs in a secondary thread, but typically that is not important. All the functions of Multiverse are threadsafe. The function indexes through the Universes. It obtains the current pixels by calling **pixels**, creates a new list of pixels by moving the last pixel to the beginning of the list, and reloads them into the Universe by calling **set_pixel_data**. This creates a "chase" pattern for the lights, which appear to move continuously from the back of the string to the front.

Note that **pixels** returns a *copy* of the Universe's data, so you cannot accidentally access the Universe's internal state.

Example 2: Subclass Multiverse and update the light strings by implementing periodic_callback

```
from pacn.multiverse import Multiverse
from pacn.universe import RGBUniverse
from pacn.utils import DEFAULT_CID_BYTES
from pacn.pac_logger import logging

IP = '127.0.0.1'   # Loopback for use with e131_sim.py

logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)

class Caneverse(Multiverse):
    def __init__(self, *x, **y) -> None:
        super().__init__(*x, **y)

        self.canes: list[RGBUniverse] = []

        wide = ["red"] * 6 + ["white"] * 9
        narrow = ["green"] * 4 + ["white"] * 11
        size = 150
        stripes = size // len(wide)
        patterns = [wide, narrow, narrow, wide]
        for i, p in enumerate(patterns, 1):
            cane = RGBUniverse(DEFAULT_CID_BYTES, i, IP, size)
            cane.set_pixel_data(p*stripes)
            self.canes.append(cane)
            self.add_universe(cane)

    def periodic_callback(self, _t: float) -> None:
        for c in self.canes:
```

```
            pixels = c.pixels()
            pixels = pixels[-1:] + pixels[:-1]
            c.set_pixel_data(pixels)

def main() -> None:
    with Caneverse() as mv:
        mv.wait_for_kb()

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        pass
```

Explanation

Example2 is very similar to Example1. The difference is how the Universes are updated. The class **Caneverse** is created as a subclass of **Multiverse**. Multiverse implements a function named **periodic_callback,** which is called once per frame. In the base class this function is just an empty stub. **Caneverse** overrides this function to update the light strings. Like the callback function **step** in Example1, this function runs in Multiverse's secondary thread.

Example 3: Update the light strings in a timing loop in main()

```
import time

from pacn.multiverse import Multiverse
from pacn.universe import RGBUniverse
from pacn.utils import DEFAULT_CID_BYTES
from pacn.pac_logger import logging

IP = '127.0.0.1'   # Loopback for use with e131_sim.py

logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)

canes: list[RGBUniverse] = []

def create_canes() -> None:
    wide = ["red"] * 6 + ["white"] * 9
    narrow = ["green"] * 4 + ["white"] * 11
    size = 150
    stripes = size // len(wide)
    patterns = [wide, narrow, narrow, wide]
    for i, p in enumerate(patterns, 1):
        cane = RGBUniverse(DEFAULT_CID_BYTES, i, IP, size)
        cane.set_pixel_data(p*stripes)
        canes.append(cane)

def step(n: int) -> None:
    c = canes[n]
    pixels = c.pixels()
    pixels = pixels[-1:] + pixels[:-1]
    c.set_pixel_data(pixels)

def main() -> None:
    create_canes()
```

```
    with Multiverse() as mv:
        for c in canes:
            mv.add_universe(c)
        i = 0
        while True:
            for n in range(4):
                if n in (1, 2) and i % 2 == 0:
                    step(n)
                step(n)
            i += 1
            time.sleep(0.1)

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        pass
```

Explanation

Example3 is likewise very similar to Example1, differing mainly in how the Universes are updated. Like Example1, **main** creates Cane objects, creates a Multiverse, and adds the canes to the multiverse. Then it enters a timing loop that iterates every 0.1 seconds by calling **time.sleep(0.1)**. Within this loop the light strings are updated; I update the two middle canes (indexes 1 and 2) half as often as the other two. This results in the “chase” pattern moving twice as fast in those cases.

In this example the pixels are manipulated in the main thread, in contrast to the other two examples. The public functions in Universe are threadsafe, so this can be done straightforwardly.

Acknowledgment

There once was a package named “sacn,” which was also a python implementation of E1.31. **PACN** has almost no code in common with **sacn**, but without this earlier package the design and implementation of **PACN** would have been significantly more difficult.